

PatchMatch e Deformable Convolutional Networks

Paper, implementazioni e risultati sperimentali

Maria Amelia Morsellino

Paper Visione Classica: PatchMatch

- problema della ricerca patch-based
- algoritmo: NNF, propagazione, random search
- strumenti di image editing strutturale
- progetto: implementazione, esperimenti e limiti

Paper Visione Deep: DCN

- limite geometrico delle CNN classiche
- deformable convolution e deformable RoI pooling
- integrazione in architetture per detection/segmentation
- progetto: classificazione, offset, robustezza e costo

PatchMatch: contesto e motivazione



Molti metodi classici di grafica computazionale modificano immagini copiando **patch** compatibili da una sorgente.

- **Image completion:** riempire una regione mancante
- **Image retargeting:** cambiare dimensione preservando struttura
- **Image reshuffling:** spostare parti dell'immagine e ricostruire lo sfondo

Il problema comune è trovare rapidamente, per moltissime patch, la patch più simile in un dominio ampio.

L'utente non può aspettare molto tempo per ogni tentativo. Il paper nasce proprio per rendere interattivi strumenti che erano già qualitativamente validi, ma troppo lenti.

Prima di PatchMatch: qualità alta, ricerca lenta

- **Texture synthesis non parametrica:** osserva ciò che circonda la zona da ricostruire e cerca un contesto simile nel resto dell'immagine. Una volta trovato, copia da lì il pixel necessario per riempire la zona mancante e continua in questo modo finché non la riempie. Si chiama non parametrica perché non va a descrivere la zona cercata attraverso un modello, ma usa direttamente il materiale presente nell'immagine.
- **Image quilting:** invece di ricostruire l'immagine un pixel alla volta, usa patch intere, che sovrappone leggermente per ridurre bordi bruschi. Anche se è più veloce del riempimento pixel per pixel, deve comunque cercare patch compatibili
- **Completion globale:** i metodi precedenti prendono decisioni una dopo l'altra, mentre la completion globale tende a ragionare sull'intera regione da ricostruire, alternando una fase di ricerca delle patch migliori a una fase di ricostruzione della zona mancante. Questa tecnica è costosa perché ad ogni iterazione deve cercare corrispondenze per moltissime patch.

Come si può notare quindi il collo di bottiglia non è la rappresentazione a patch in sé: è il calcolo ripetuto del nearest neighbor per ogni patch.

PatchMatch non sostituisce l'intera pipeline di editing, ma sostituisce il pezzo più costoso, ossia l'algoritmo che calcola le corrispondenze tra patch.

Nearest-Neighbor Field e costo brute force

Dato un dominio di patch in A e uno in B , il **Nearest-Neighbor Field** è una funzione

$$f : A \rightarrow \mathbb{R}^2, \quad f(a) = b - a$$

Per ogni posizione a in A vogliamo trovare la posizione b in B in cui si trova la patch più simile.

- $f(a)$ non memorizza direttamente la patch b : memorizza l'**offset**.
- Il campo finale è quindi una mappa di vettori 2D.

Se:

- m = numero di pixel nella patch
- M = numero di posizioni patch valide
- ogni patch di A viene confrontata con tutte le patch di B

il costo brute force è:

$$O(mM^2)$$

Questa formula spiega perché il problema esplode rapidamente. Aumentando la risoluzione, non cresce solo il numero di patch: cresce anche il numero di confronti tra tutte le coppie possibili.

Intuizioni fondamentali di PatchMatch

1. Random sampling

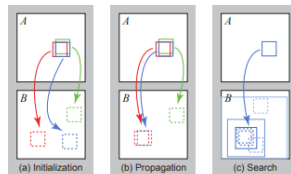
Una inizializzazione casuale è rumorosa, ma su molte patch è probabile che alcuni offset siano già buoni. Questi match diventano **seed**.

2. Spazio degli offset

Il metodo evita strutture complesse nello spazio ad alta dimensionalità delle patch e lavora direttamente nello spazio 2D degli spostamenti.

3. Coerenza naturale

Patch vicine tendono ad avere corrispondenze vicine. Un offset buono può quindi propagarsi ai vicini.



Fonte: PatchMatch.

La forza del metodo sta nella combinazione: il caso trova qualche soluzione utile, la propagazione la diffonde, e la ricerca casuale evita che il processo resti troppo locale.

Schema operativo dell'algoritmo

Input

- immagine o feature sorgente A
- immagine o feature target B
- metrica di distanza tra patch $D(\cdot, \cdot)$

Output

- NNF: offset $f(x, y)$ per ogni posizione
- NND: distanza associata al match corrente

Iterazione tipica

- 1 inizializzazione casuale degli offset
- 2 propagazione
- 3 random search intorno al match corrente
- 4 ordine invertito nelle iterazioni successive

Il risultato non è necessariamente il nearest neighbor esatto, ma un'approssimazione molto buona in tempi pratici. Per l'editing è più importante avere una soluzione accettabile subito che una soluzione esatta troppo tardi.

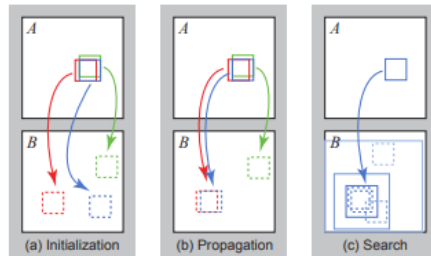
Propagazione: sfruttare i vicini già visitati

Nella scansione da sinistra a destra e dall'alto in basso, per aggiornare $f(x, y)$ si confrontano tre candidati:

$$C(x, y) = \{f(x, y), f(x - 1, y), f(x, y - 1)\}$$

$$f(x, y) = \arg \min_{v \in C(x, y)} D(v)$$

Nelle iterazioni alternate si inverte il verso della scansione.



Fonte: PatchMatch.

Quindi se il vicino ha trovato una buona zona sorgente, provo a riusare la sua ipotesi anche per la patch corrente. Questo ha senso perché le immagini naturali solitamente sono localmente regolari.

Random search: esplorazione a raggio decrescente

La propagazione è locale. Per uscire da minimi locali, PatchMatch testa offset casuali in finestre sempre più piccole:

$$u_i = v_0 + w\alpha^i R_i$$

- $v_0 = f(x, y)$: offset corrente
- w : raggio iniziale
- $\alpha \in (0, 1)$: fattore di riduzione
- R_i : vettore casuale uniforme in $[-1, 1]^2$

Ruolo della random search

Prima esplora lontano dal match corrente; poi raffina localmente. Così il metodo resta semplice, ma non dipende solo dai vicini.

Questa fase è il complemento della propagazione. La propagazione diffonde buone soluzioni locali; la random search introduce salti più ampi e quindi riduce il rischio di rimanere bloccati.

Editing tools e bidirectional similarity

Una volta disponibile un NNF veloce, PatchMatch può essere usato dentro una pipeline di sintesi basata su **bidirectional similarity**

$$d_{BDS}(S, T) = d_{complete}(S, T) + d_{cohere}(S, T)$$

- **Completezza** $S \rightarrow T$: ogni patch della sorgente dovrebbe comparire anche nel target. Evita di perdere contenuti importanti.
- **Coerenza** $T \rightarrow S$: ogni patch del target dovrebbe avere una patch simile nella sorgente. Evita contenuti inventati o artefatti.

La Bidirectional Similarity è quindi il criterio che si vuole ottimizzare, **Search + Vote** è il procedimento usato per farlo.

- **Search**: con PatchMatch si trovano, per ogni patch del target, le patch più simili nella sorgente, e viceversa se si usa la bidirectional similarity. In pratica si stabilisce da dove prendere il contenuto.
- **Vote**: le patch trovate propongono i valori dei pixel del nuovo target. Poiché le patch si sovrappongono, ogni pixel riceve più proposte, che vengono combinate, spesso con una media o una media pesata. Poi Search e Vote vengono ripetuti.

Vincoli geometrici e limiti

Vincoli geometrici

PatchMatch confronta patch localmente simili, ma può essere guidato per rispettare strutture più ampie:

- **Continuità di linee:** un bordo o una linea deve proseguire coerentemente dentro una regione mancante
- **Trasformazioni controllate:** alcune regioni possono seguire traslazioni, cambiamenti di scala o trasformazioni prospettiche
- **Stima robusta:** metodi come RANSAC permettono di stimare il modello geometrico ignorando le corrispondenze errate

Limiti principali

- **Ambiguità locali:** texture ripetitive possono produrre molti match visivamente simili ma geometricamente diversi
- **Artefatti:** patch incompatibili possono causare ghosting, bordi doppi o feathering
- **Ottimizzazione locale:** l'algoritmo può convergere verso un minimo locale non soddisfacente
- **Assenza di semantica:** non comprende oggetti, simmetrie o relazioni globali tra parti lontane

Progetto PatchMatch: obiettivi

Il progetto nasce con l'obiettivo di verificare sperimentalmente il funzionamento di PatchMatch. Si osserva:

- come migliorano progressivamente le corrispondenze tra immagini
- quanto le soluzioni ottenute siano accurate ed efficienti
- come PatchMatch possa essere impiegato nella ricostruzione di immagini e nell'inpainting
- in quali situazioni il metodo funziona bene e quali sono invece i suoi limiti

Esperimento 1 - Dalle corrispondenze alla ricostruzione

Obiettivo: verificare se PatchMatch riesce a ricostruire l'immagine A utilizzando contenuto prelevato dalle patch più simili trovate nell'immagine B.

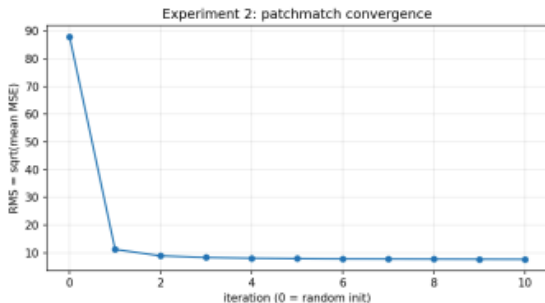


- A definisce la struttura dell'immagine da ricostruire
- B è il dominio in cui vengono cercate le patch compatibili
- le due immagini inferiori sono ricostruzioni di A, non trasformazioni di A in B

Il primo esperimento serve come verifica generale del funzionamento dell'algoritmo. Partiamo da due immagini della stessa scena, ma con alcune differenze di posizione e contenuto. PatchMatch cerca, per ogni zona dell'immagine A, una zona simile nell'immagine B. Visto che la ricostruzione assomiglia all'originale, significa che le corrispondenze trovate sono complessivamente coerenti.

Esperimento 2 - Quanto velocemente migliora PatchMatch

Obiettivo: osservare quanto rapidamente PatchMatch riesce a passare da corrispondenze inizialmente casuali a corrispondenze più accurate.



Risultato

- all'inizio l'errore è molto elevato, perché le corrispondenze sono casuali
- dopo una sola iterazione il RMS scende
- nelle iterazioni successive il miglioramento continua, ma diventa progressivamente più piccolo

L'esperimento ha funzionato come previsto. La maggior parte del miglioramento avviene già nella prima iterazione, quando le buone corrispondenze iniziano a diffondersi nell'immagine. Dopo poche iterazioni la curva si stabilizza, mostrando che non è necessario ripetere l'algoritmo molte volte per ottenere un risultato utile.

Esperimento 3 - Ricostruzione di una regione mancante

Obiettivo: verificare se PatchMatch può essere utilizzato per ricostruire una parte rimossa dell'immagine, copiando contenuto compatibile dalle regioni ancora disponibili.



Confronto tra immagine originale, maschera, immagine danneggiata con hole rettangolare e risultato finale dell'inpainting.

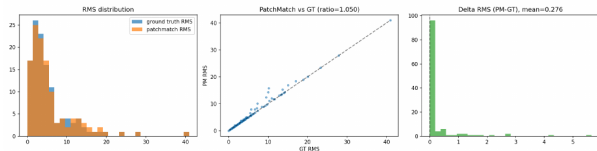
- è stata rimossa una regione rettangolare dalla fotografia
- la ricostruzione è iniziata da una versione a bassa risoluzione
- il risultato è stato poi raffinato progressivamente alle risoluzioni maggiori

In questo caso l'esperimento ha prodotto una ricostruzione complessivamente plausibile. Lavorare inizialmente a bassa risoluzione ha permesso di definire una struttura generale del contenuto mancante, mentre i livelli successivi hanno migliorato i dettagli. Il risultato funziona perché intorno al buco erano presenti zone abbastanza simili da poter essere riutilizzate.

Esperimento 4 - Confronto con la ricerca esatta

Obiettivo: capire quanto le corrispondenze approssimate trovate da PatchMatch siano vicine a quelle ottenute controllando tutte le possibilità.

Nota: la ricerca esaustiva è stata applicata soltanto a una parte dei punti dell'immagine, perché confrontare tutte le patch sarebbe stato troppo costoso.



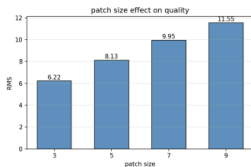
Risultato

- PatchMatch produce un errore medio soltanto di circa il 5% superiore
- le corrispondenze ottenute restano generalmente vicine a quelle esatte

L'esperimento mostra che PatchMatch ha funzionato bene come metodo approssimato. Non trova necessariamente la migliore corrispondenza possibile in ogni posizione, ma nel test si avvicina molto alla soluzione della ricerca completa, evitando il costo di controllare tutte le combinazioni.

Esperimento 5 - Effetto della dimensione delle patch

Obiettivo: capire come cambia il comportamento di PatchMatch quando si confrontano porzioni più piccole o più grandi dell'immagine.



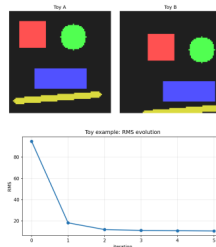
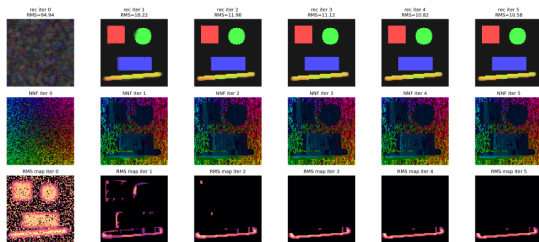
Confronto visivo tra l'immagine originale A e le ricostruzioni ottenute con diverse dimensioni di patch tramite media di patch sovrapposte. Ogni pannello riporta la RMS corrispondente.

- le patch piccole riescono a trovare più facilmente somiglianze locali
- le patch grandi considerano una porzione più ampia dell'immagine, ma impongono condizioni più rigide
- nel test fatto qui, aumentando la dimensione delle patch aumentano sia il RMS sia il tempo di esecuzione
- la dimensione più piccola ha ottenuto il risultato numericamente migliore

L'esperimento mostra che non è sempre vantaggioso utilizzare patch più grandi. In questo caso le patch piccole hanno prodotto corrispondenze migliori, mentre quelle grandi hanno faticato a trovare regioni completamente compatibili. La dimensione ideale dipende quindi dal contenuto dell'immagine e dal tipo di struttura che si vuole conservare.

Esperimento 6 - Evoluzione passo dopo passo

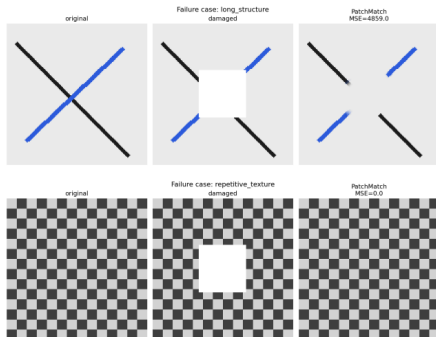
Obiettivo: rendere visibile il funzionamento di PatchMatch attraverso un esempio semplice e controllato.



L'immagine B contiene le stesse forme principali dell'immagine A, ma spostate. Esiste quindi una struttura di corrispondenze relativamente semplice, che dovrebbe essere recuperata tramite offset coerenti. All'iterazione zero il nearest-neighbor field è casuale, infatti la ricostruzione è piena di rumore. Dopo la prima iterazione la RMS scende notevolmente e le forme principali diventano già riconoscibili. Le tre righe mostrano aspetti diversi: la prima mostra la ricostruzione ottenuta dal campo di offset corrente. La seconda mostra il nearest-neighbor field in HSV. La terza mostra la mappa dell'errore. All'inizio l'errore è diffuso in quasi tutta l'immagine. Poi si concentra soprattutto sui bordi e sulle parti più difficili.

Esperimento 7 - Casi in cui PatchMatch fallisce

Obiettivo: verificare il comportamento del metodo in situazioni volutamente difficili, nelle quali le sole somiglianze locali potrebbero non essere sufficienti.



Risultati

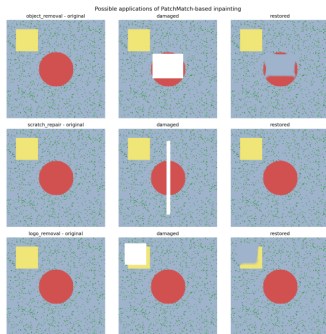
- nella scacchiera esistono molte patch quasi identiche, quindi la soluzione può essere ambigua, anche se viene ricostruita in maniera plausibile
- nel caso delle linee lunghe il risultato è peggiore, poiché manca un pezzo fondamentale

L'esperimento mostra chiaramente un limite del metodo. PatchMatch confronta soprattutto informazioni locali e non comprende la struttura globale della scena. Per questo può ricostruire bene una texture ripetitiva, ma può fallire quando deve capire che due segmenti separati appartengono alla stessa linea e devono essere collegati.

Esperimento 8 - Applicazioni pratiche

Obiettivo: valutare PatchMatch su alcune semplici operazioni di restauro e rimozione di elementi da un'immagine.

Applicazioni testate: object removal, scratch repair e logo removal.



Risultati

- la riparazione del graffio è il caso riuscito meglio
- la rimozione dell'oggetto lascia alcuni artefatti
- la rimozione del logo è soltanto parziale

I risultati sono quindi misti. PatchMatch funziona bene quando la regione mancante è sottile e può essere ricostruita usando immediatamente il contenuto circostante, come nel caso del graffio. Ha invece più difficoltà quando deve eliminare una parte ampia di un oggetto o ricostruire una forma precisa senza avere nel contesto patch realmente compatibili.

Deformable Convolutional Networks

Limite geometrico delle CNN standard

Le CNN standard campionano feature su griglie fisse:

- stessa geometria del kernel in ogni posizione
- stesso pattern anche se scala, posa o forma cambiano
- RoI pooling divide le regioni in bin regolari, senza seguire la forma degli oggetti

Problema

La rete può imparare robustezza tramite dati e pesi, ma il meccanismo di lettura resta geometricamente rigido.

DCN interviene nel punto in cui la CNN legge la feature map. L'idea è semplice: oltre ai pesi del filtro, impara anche piccoli spostamenti delle posizioni da cui campionare, ossia gli offset.

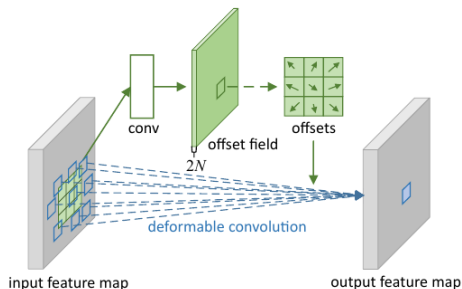


Figure 2: Illustration of 3×3 deformable convolution.

Fonte: DCN.

Convoluzione standard e deformabile

Standard convolution

$$y(p_0) = \sum_{p_n \in R} w(p_n) x(p_0 + p_n)$$

- R : griglia del kernel
- $w(p_n)$: pesi del filtro
- campionamento rigido

Deformable convolution

$$y(p_0) = \sum_{p_n \in R} w(p_n) x(p_0 + p_n + \Delta p_n)$$

- Δp_n : offset 2D appreso
- se $\Delta p_n = 0$, si torna alla convoluzione standard
- il supporto spaziale del kernel diventa adattivo

La parte nuova non sostituisce i pesi: li affianca. I pesi decidono quanto pesare ogni campione; gli offset decidono dove andare a leggere quel campione.

Offset e interpolazione bilineare

Per ogni punto del kernel, la rete deve imparare uno spostamento orizzontale e uno verticale.

Con un kernel $k \times k$ abbiamo $N = k^2$ punti, quindi il ramo convoluzionale che predice gli offset produce:

$$2N = 2k^2$$

canali, cioè una coppia $(\Delta x, \Delta y)$ per ciascun punto del kernel.

Gli spostamenti appresi non devono necessariamente portare a coordinate intere. Un punto del kernel può quindi trovarsi tra più celle della feature map.

In questo caso il valore da campionare viene calcolato tramite **interpolazione bilineare**: si combinano i valori dei quattro punti più vicini, assegnando un peso maggiore a quelli più vicini alla posizione richiesta.

Permette ai punti di campionamento di muoversi in modo continuo e rende possibile aggiornare gli offset tramite la normale backpropagation.

Deformable RoI pooling

RoI pooling standard divide una RoI in $k \times k$ bin.
DCN aggiunge un offset per ogni bin:

$$y(i, j) = \frac{1}{n_{ij}} \sum_{p \in \text{bin}(i, j)} x(p_0 + p + \Delta p_{ij})$$

Gli offset sono normalizzati e scalati rispetto alla RoI

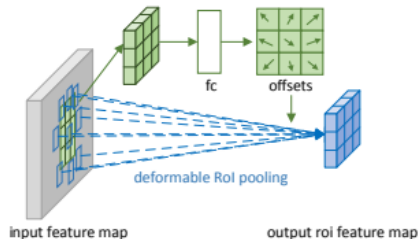


Figure 3: Illustration of 3×3 deformable RoI pooling.

Fonte: DCN.

Nel detection, il problema non è solo estrarre feature migliori, ma anche allineare meglio la griglia di pooling alle parti effettive dell'oggetto, soprattutto se l'oggetto è deformabile.

Integrazione in architetture complete

I moduli deformabili mantengono input e output delle controparti standard, quindi possono essere usati come **drop-in replacement**.

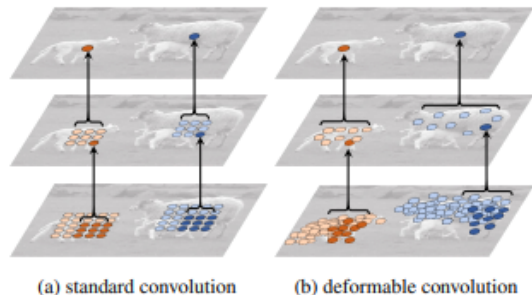
- Deformable convolution negli ultimi layer della backbone.
- Deformable RoI pooling nella parte region-based dei detector.
- Configurazione tipica del paper: tre layer deformabili negli strati alti.

Vantaggio fondamentale

DCN non richiede una rete completamente nuova: modifica moduli locali già presenti.

Questo è un aspetto molto importante del paper: DCN non propone solo una intuizione teorica, ma moduli sostituibili dentro architetture già.

Receptive field adattivo



Fonte: DCN.

- Convoluzione standard: pattern fisso in ogni posizione.
- Deformable convolution: punti spostati in funzione del contenuto.
- Layer deformabili impilati: campo recettivo con scala e forma variabili.

Effective dilation

Misura media della distanza tra punti adiacenti campionati dal filtro deformabile.

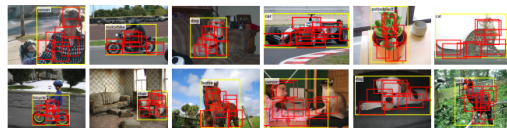
Questa è la differenza rispetto alla dilated convolution: la dilatazione fissa allarga il campo recettivo sempre nello stesso modo; DCN lo adatta localmente all'oggetto.

Evidenza qualitativa dal paper



I punti rossi mostrano posizioni di campionamento apprese su oggetti di scala diversa.

La lettura qualitativa è coerente con l'idea del metodo: i campioni non restano sulla griglia originaria, ma tendono a distribuirsi in modo più utile rispetto alla forma dell'oggetto.



I bin deformabili si spostano verso parti informative dell'oggetto.

Risultati del paper e trade-off

- Deformable convolution migliora rispetto ai layer standard e alle alternative con dilatazione fissa.
- Deformable RoI pooling migliora soprattutto la localizzazione più severa.
- Usare entrambi i moduli produce ulteriori guadagni.
- Aumentare il numero di layer deformabili migliora fino a una saturazione.
- L'overhead di parametri e runtime è contenuto rispetto ai guadagni nei task originali.

Conclusion

Il vantaggio non deriva solo dall'aumento di capacità parametrica, ma dall'introduzione di un meccanismo più adatto a modellare trasformazioni geometriche locali.

La geometria non viene solo appresa implicitamente dai pesi, ma viene inserita nel meccanismo di campionamento, e questo aiuta nei task in cui scala, posa e deformazione sono centrali.

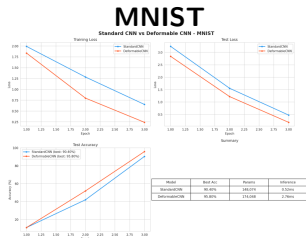
Il progetto trasferisce il contributo centrale di DCN in una pipeline **CPU-oriented** di classificazione.

- StandardCNN vs DeformableCNN
- MNIST e FashionMNIST
- offset inizializzati a zero e tracciati esplicitamente

L'obiettivo non è replicare i benchmark grandi del paper, ma isolare il meccanismo: vedere se una rete deformabile piccola migliora la classificazione, come si muovono gli offset e quanto costa il modulo su CPU.

Esperimento 1 - Prestazioni e costo computazionale

Obiettivo: verificare se il campionamento deformabile migliora la classificazione rispetto a una CNN standard, utilizzando le stesse condizioni di addestramento su MNIST e FashionMNIST.



Risultato: la DeformableCNN ottiene prestazioni migliori su entrambi i dataset, ma richiede tempi di addestramento e inferenza molto superiori sulla CPU.

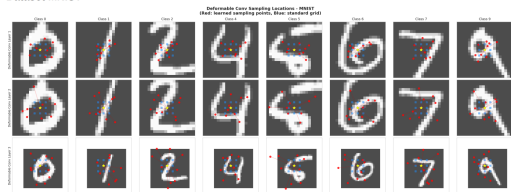
L'esperimento mostra che rendere adattive le posizioni di campionamento può migliorare la classificazione. Il vantaggio non compare soltanto sui digit di MNIST, ma anche sugli oggetti più complessi di FashionMNIST. Questo risultato deve però essere letto insieme al costo computazionale: nella pipeline utilizzata, la convoluzione deformabile è molto più lenta della convoluzione standard, soprattutto durante l'addestramento su CPU.

Esperimento 2 - Cosa imparano gli offset

Obiettivo: osservare se gli offset appresi modificano realmente la griglia del kernel e capire verso quali regioni dell'immagine tendono a spostare i punti di campionamento.

MNIST

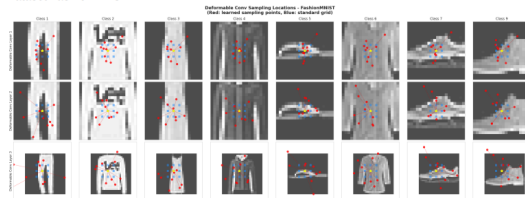
Dataset MNIST



Nei digit, i punti deformati tendono a seguire tratti, curve e parti informative delle cifre.

FashionMNIST

Dataset FashionMNIST

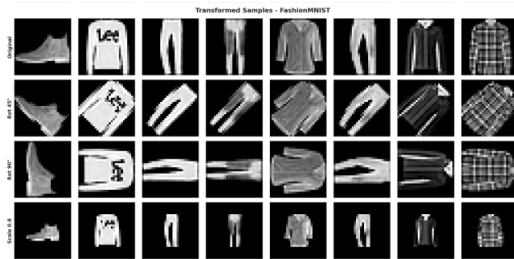
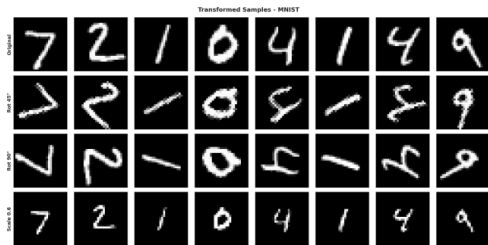


Negli oggetti più complessi, gli spostamenti interessano spesso bordi, contorni e parti della silhouette.

Questo esperimento serve a verificare che la rete utilizzi davvero la possibilità di deformare il campionamento. Le visualizzazioni mostrano che i punti non restano sulla griglia regolare iniziale, ma tendono a distribuirsi attorno a strutture utili, come i tratti dei numeri o i contorni degli indumenti.

Esperimento 3 - Robustezza a trasformazioni geometriche

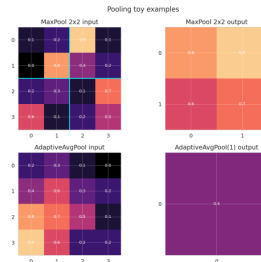
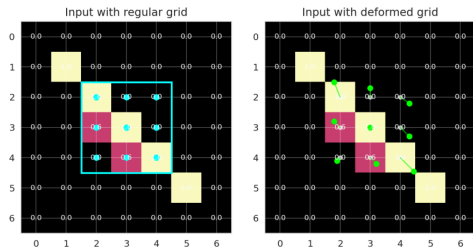
Obiettivo: verificare come si comportano la CNN standard e quella deformabile quando le immagini di test vengono ruotate, ridimensionate o deformate, senza riaddestrare i modelli su queste trasformazioni.



L'esperimento ha trovato che il campionamento adattivo è molto utile quando cambia la dimensione dell'oggetto, poiché la rete deformabile può restringere o ampliare la distribuzione dei punti per seguire meglio il segnale. Il vantaggio è più limitato con rotazioni e shear estremi, perché modificare localmente la griglia non rende automaticamente la rete invariante a qualsiasi trasformazione geometrica.

Esperimento 4 - Toy test su sampling e pooling

Obiettivo: mostrare con esempi visivi che cosa cambia concretamente nel campionamento tra una convoluzione standard e una deformabile.

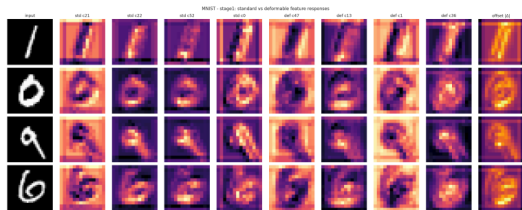


Questo esperimento non serve a misurare l'accuratezza, ma a rendere visibile il funzionamento del metodo. Nella convoluzione standard il kernel legge sempre gli stessi punti relativi rispetto al centro, anche quando il segnale è inclinato o curvo. Nella convoluzione deformabile, invece, i punti possono spostarsi e il pattern di campionamento può adattarsi alla struttura presente nella feature map. Le altre visualizzazioni mostrano poi come il pooling riduce e riassume l'informazione

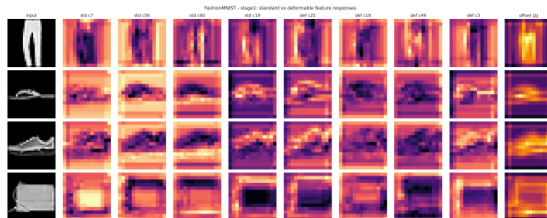
Esperimento 5 - Feature interne: stage 1

Obiettivo: confrontare le rappresentazioni prodotte dalle due reti nei primi livelli e osservare se la deformabilità modifica già il modo in cui vengono descritti i pattern locali.

MNIST



FashionMNIST



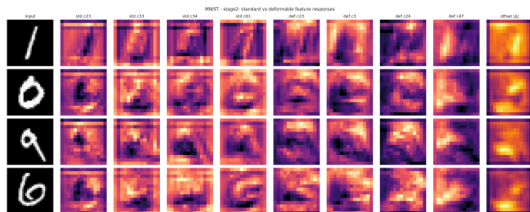
Risultato: in questo stadio entrambe le reti rappresentano soprattutto bordi, tratti e contrasti; alcune feature deformabili appaiono però più aderenti alla geometria locale dell'immagine.

Nel primo stage le rappresentazioni sono ancora molto semplici e vicine all'aspetto locale dell'input. Su MNIST vengono evidenziati soprattutto gli stroke delle cifre, mentre su FashionMNIST emergono bordi e profili degli oggetti. Le differenze tra le due reti sono ancora limitate, ma alcune attivazioni deformabili sembrano seguire meglio le linee e i contorni presenti nell'immagine.

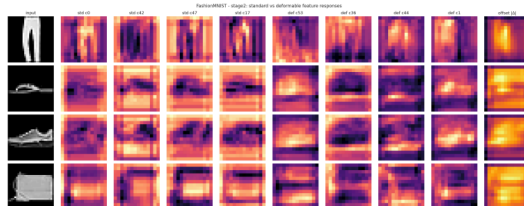
Esperimento 5 - Feature interne: stage 2

Obiettivo: osservare come cambia il confronto nei livelli intermedi, dove le feature iniziano a rappresentare parti e strutture più complesse rispetto ai semplici bordi.

MNIST



FashionMNIST



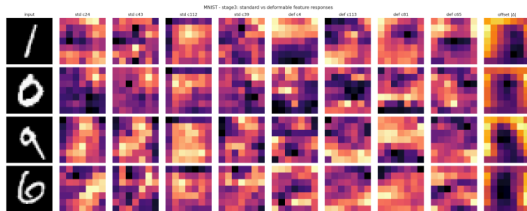
Risultato: le differenze diventano più leggibili e le attivazioni deformabili tendono spesso a concentrarsi sulle parti più caratteristiche dell'oggetto.

Nel secondo stage le feature non descrivono più soltanto bordi elementari, ma iniziano a rappresentare combinazioni di forme. Su MNIST possono concentrarsi su curve e tratti distintivi delle cifre; su FashionMNIST su contorni e parti strutturali degli indumenti.

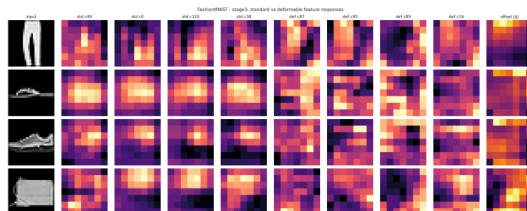
Esperimento 5 - Feature interne: stage 3

Obiettivo: verificare se l'effetto della deformabilità continua a essere presente anche negli strati più profondi, dove le rappresentazioni sono più astratte e compresse.

MNIST



FashionMNIST



Risultato: le mappe sono meno interpretabili visivamente, ma la rete deformabile continua a produrre risposte differenti in funzione dell'oggetto e non semplicemente uniformi.

Nel terzo stage la risoluzione è molto ridotta e non è più possibile collegare facilmente ogni attivazione a un bordo o a una parte precisa dell'immagine. Tuttavia, le feature continuano a cambiare in funzione dell'input e gli effetti della deformabilità non scompaiono.

Confronto finale: PatchMatch e DCN

PatchMatch

- metodo classico, patch-based
- ricerca approssimata di corrispondenze
- sfrutta coerenza spaziale e random search
- interpretabile, utile per editing e inpainting
- limite: coerenza globale e semantica

DCN

- metodo deep
- apprende dove campionare le feature
- introduce geometria adattiva nei layer
- utile per detection, segmentation e classificazione
- limite: maggiore complessità computazionale

PatchMatch mostra come un algoritmo classico possa rendere praticabile l'editing strutturale accelerando il matching tra patch. DCN mostra come una rete deep possa migliorare la modellazione geometrica rendendo deformabile il campionamento delle feature.